Security Analysis and Countermeasures for Blockchain Simulation

October 12, 2025

1 Introduction

This report analyzes the security aspects of a blockchain simulation implemented in Python using SimPy and Flask. The simulation models a network with six nodes (five honest, one malicious) and focuses on a double-spending attack where the malicious node (node 5) attempts to nullify a transaction to a victim node by replacing the shared blockchain with a private chain. The report evaluates the effectiveness of the double-spending attack strategy, proposes security enhancements such as transaction confirmations and fork resolution, and provides recommendations to mitigate double-spending vulnerabilities.

2 Effectiveness of Attack Strategies

2.1 Double-Spending Attack Mechanism

The simulation implements a double-spending attack where the malicious node (node 5) initiates two transactions simultaneously:

- **Public Transaction**: From node 5 to a victim node (e.g., node 4) for an amount (e.g., 300.0 units), added to the shared blockchain.
- **Malicious Transaction**: From node 5 to the attacker's wallet (e.g., `attacker_wallet_5`) for the same amount, mined secretly on a private chain.

Both transactions share the same timestamp, ensuring simultaneity. The public transaction is added to the shared blockchain's pending transactions and broadcast to all nodes, while the malicious transaction is mined into a block on the private chain. If the attack succeeds, the private chain replaces the shared blockchain, nullifying the victim's transaction.

2.2 Success Conditions

The attack's success depends on:

- **Hash Rate**: The malicious node's hash rate (6.0) must exceed the total hash rate of honest nodes (5.0, with each honest node at 1.0).
- **Network Delay**: The malicious node's network delay (0.1 seconds) must be lower than the average honest node delay (0.4 seconds, calculated as $(0.3 + 0.4 + 0.35 + 0.45 + 0.5)/5$).

In the simulation, these conditions are met, leading to a successful attack, as evidenced by the terminal output where the shared blockchain is replaced with the private chain containing only the malicious transaction.

2.3 Attack Effectiveness

The double-spending attack is highly effective in the current setup due to:

- **Favorable Conditions**: The malicious node's superior hash rate and lower delay guarantee success, as the private chain can be mined and broadcast faster than honest nodes can mine the public transaction.

- **Lack of Confirmations**: The simulation does not require multiple block confirmations, allowing the private chain to replace the shared chain immediately.
- **Instant Mining**: With low or zero difficulty (though the output suggests non-zero difficulty), mining is fast, enabling rapid execution of the attack.

The terminal output confirms the attack's success: the public transaction (node 5 node 4) is nullified, and the malicious transaction (node 5 `attacker_wallet_5`) is included in the blockchain, ensuring the attacker retains the coins.

3 Security Enhancements

To mitigate the double-spending attack, two enhancements are proposed and implemented: transaction confirmations and fork resolution.

3.1 Transaction Confirmations

Requiring multiple confirmations (additional blocks mined after the block containing a transaction) makes it harder for an attacker to replace the blockchain. The simulation can be modified to enforce a confirmation threshold (e.g., 3 confirmations) before a transaction is considered final.

Implementation:

- Add a `confirmation_threshold` parameter to the `Blockchain` class.
- Modify the `add_block` method to check if a transaction has sufficient confirmations (i.e., the number of blocks mined after its inclusion meets the threshold).
- Update the Flask application to display confirmation status for transactions.

This ensures that the victim's transaction (e.g., node 5 node 4) must be followed by additional blocks, increasing the computational effort required for the attacker to build a longer private chain.

3.2 Fork Resolution

Fork resolution ensures that when multiple chains exist (e.g., shared and private), the network adopts the longest valid chain. The current simulation replaces the shared chain with the private chain if the attack succeeds, but it lacks robust validation.

Implementation:

- Add a `resolve_fork` method to the `Blockchain` class to compare chain lengths and validate the longest chain's integrity (e.g., checking hashes and transactions).
- Update `perform_double_spending_attack` to invoke `resolve_fork` when broadcasting the private chain.
- Ensure all nodes verify the new chain's validity before adopting it.

This mechanism ensures that an attacker must mine multiple blocks to outpace the honest chain, increasing the attack's difficulty.

4 Mitigating Double-Spending Vulnerabilities

To further reduce double-spending vulnerabilities, the following countermeasures are recommended:

1. **Increase Confirmation Requirements**: Set a high confirmation threshold (e.g., 6 blocks) for high-value transactions. This forces the attacker to mine multiple blocks, which is computationally expensive, especially if honest nodes collectively have a higher hash rate.
2. **Enhance Network Synchronization**: Reduce network delays for honest nodes or implement faster consensus protocols to minimize the attacker's delay advantage.

3. **Implement Transaction Locks**: Introduce a mechanism to lock funds until confirmations are met, preventing the attacker from reusing the same coins in a malicious transaction.
4. **Monitor Malicious Activity**: Add detection mechanisms to flag nodes that frequently broadcast conflicting transactions, allowing the network to isolate or penalize malicious nodes.
5. **Increase Honest Hash Rate**: Distribute hash power more evenly among honest nodes to make it harder for a single malicious node to dominate.

5 Conclusion

The double-spending attack in the blockchain simulation is highly effective due to the malicious node's superior hash rate and low network delay, as demonstrated by the successful replacement of the shared blockchain with a private chain containing a malicious transaction. Implementing transaction confirmations and fork resolution significantly enhances security by requiring attackers to mine multiple blocks and ensuring only valid chains are adopted. Additional countermeasures, such as increasing confirmations, improving network synchronization, and monitoring malicious activity, can further mitigate double-spending vulnerabilities. These enhancements make the blockchain more resilient while maintaining its core functionality.